8.1.3 Legacy and lessons

Although XACML is seldom used in new systems today, its influence is still felt. The architecture it proposed—decoupling enforcement from decision logic—is reflected in policy engines like OPA and Cedar. Its detailed approach to attribute-based logic helped shape ABAC models, even if its syntax became a cautionary tale.

At ACME, XACML never made it out of the data center. The document archive where it was used remains a legacy system, with the original policies largely untouched—too fragile and obscure for most developers to safely update. But the experience shaped the company's thinking: centralized policy was powerful, but only if the language was accessible and flexible.

8.2 OPA and Rego: Policy for cloud-native systems

As ACME's infrastructure evolved toward microservices and Kubernetes, the identity team found that traditional role-based access and legacy systems like XACML couldn't keep up. They needed a way to enforce consistent policies across APIs, CI/CD pipelines, container deployments, and more. That search led them to the Open Policy Agent (OPA, https://www.openpolicyagent.org)—a general-purpose policy engine—and its policy language, Rego.

OPA was designed for modern, distributed systems. It provides a lightweight decision engine that can be embedded into applications, gateways, and controllers. Rather than focus solely on access control, OPA treats policy more broadly: it can validate Kubernetes resources, restrict API inputs, enforce admission control, and—yes—make authorization decisions. This flexibility made it a favorite in the cloud-native ecosystem.

OPA evaluates policies written in Rego, a declarative, logic-based language inspired by Datalog. Rego lets developers write rules that define what is allowed or denied, using JSON data structures as inputs. A typical Rego policy reads more like code than configuration, which makes it powerful but sometimes difficult to learn.